

Hooks

Lifecycle 자동화 메커니즘

5 가지 hook type, exit code 제어, JSON STDIN 통신

agentSpawn, userPromptSubmit, preToolUse, postToolUse, stop

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



Agenda

Chapter 15, Hooks

- | | |
|----|--|
| 01 | Hooks 개요와 활용
Lifecycle 자동화, 5 가지 활용 영역 |
| 02 | 5 가지 Hook Type
agentSpawn, userPromptSubmit, preToolUse 외 |
| 03 | Hook 동작 방식
JSON STDIN, STDOUT 컨텍스트 주입 |
| 04 | Exit Code 시스템
0 성공, 2 차단, 기타 경고 |
| 05 | Tool Matcher 패턴
canonical names, aliases, MCP namespace |
| 06 | Timeout 과 Caching
기본 30초, cache_ttl_seconds 옵션 |
| 07 | 활용 예시와 Best Practices
Audit log, 자동 포맷, 보안 검증 |

Hooks 개요와 활용

LIFECYCLE AUTOMATION TRIGGERS

HOOKS OVERVIEW

Hooks 는 agent lifecycle 과 tool 실행 중 특정 시점에 명령을 자동 실행하는 메커니즘.

Agent JSON 의 hooks 필드에 정의하며, security validation, logging, formatting, context gathering 등 다양한 사용자 정의 동작을 가능.

VALIDATION

보안 검증

PreToolUse 로 위험 명령 사전 차단.

화이트리스트, 정책 강제 적용에 활용.

LOGGING

Audit log 기록

모든 shell 명령, AWS 호출을 파일에 기록.
컴플라이언스 추적과 감사에 필수.

FORMATTING

자동 포매팅

PostToolUse 로 파일 수정 후 prettier, cargo fmt 자동 실행.
일관 스타일 유지.

CONTEXT

컨텍스트 자동 수집

AgentSpawn 으로 git status, 최근 변경 자동 수집.
Agent 에 컨텍스트 주입.

hooks

Agent 필드 이름

5 시점

Lifecycle 지원

STDIN/OUT

통신 방식

Automation

핵심 가치

5 가지 Hook Type

AGENT LIFECYCLE TRIGGER POINTS

FIVE HOOK TYPES

Agent lifecycle 의 5 가지 시점에 hook 을 정의할 수 있음.

Activation 부터 prompt 제출, tool 실행 전후, 응답 완료까지 모든 단계를 cover.

Hook Type	트리거 시점
agentSpawn	Agent 활성화 시점, tool context 없음
userPromptSubmit	사용자 prompt 제출 시점, prompt 필드 포함
preToolUse	Tool 실행 전, exit 2 로 차단 가능
postToolUse	Tool 실행 후, tool_response 결과 접근
stop	Assistant 응답 완료 시점, matcher 미사용

5 type

Lifecycle 시점

Spawn

Activation 시점

Pre/Post

Tool 전후 hook

Stop

응답 완료 시점

Hook 동작 방식

JSON STDIN, STDOUT INJECTION

COMMUNICATION MECHANISM

Hooks 는 JSON 형식 event 를 STDIN 으로 수신하고, STDOUT 출력은 agent context 에 자동 주입.

Tool 관련 hook 은 tool_name, tool_input, tool_response (PostToolUse 만) 같은 추가 필드를 받음.

INPUT, STDIN

JSON Event 수신

hook_event_name, cwd, session_id 가 기본 필드.

Tool hook 은 tool_name, tool_input 추가.

OUTPUT, STDOUT

컨텍스트 자동 주입

AgentSpawn 과 userPromptSubmit 은 STDOUT 이 context 추가.

다른 hook 은 캡처만.

ERROR, STDERR

오류 메시지 전달

비정상 exit 시 사용자에게 경고 표시.

PreToolUse exit 2 는 STDERR 가 LLM 에 전달.

STDIN

Event 입력 채널

STDOUT

컨텍스트 출력

JSON

Event 형식

STDERR

경고 전달 채널

Exit Code 시스템

0 SUCCESS, 2 BLOCK, OTHER WARNING

EXIT CODE BEHAVIOR

Hook 의 exit code 가 동작을 결정. 0 은 성공, 2 는 PreToolUse 에서 차단, 기타는 경고 후 진행.

PreToolUse 만 차단 기능이 있으며 PostToolUse 는 이미 실행된 상태.

Exit Code	Behavior
0	성공, STDOUT 처리는 hook 타입에 따라 결정
2	PreToolUse 만, tool 실행 차단, STDERR 가 LLM 에 전달
기타	Hook 실패, STDERR 가 사용자에게 경고 표시, 실행은 진행

0

성공 코드

2

차단 코드

PreToolUse

차단 가능 hook

Warning

기타 코드 동작

Tool Matcher 패턴

WHICH TOOLS TRIGGER THE HOOK

TOOL MATCHING

preToolUse 와 postToolUse hook 은 matcher 필드로 어떤 tool 에 적용될지 지정.

Canonical name, alias, MCP namespace, wildcard 모두 지원. matcher 미지정 시 모든 tool 에 적용.

CANONICAL

정식 tool 이름

fs_read, fs_write,
execute_bash, use_aws.
내부 tool 시스템의 정식 이름.

ALIASES

단축 별칭 이름

read, write, shell, aws
등 짧은 이름.
Canonical 과 동등하게 동작.

MCP

MCP namespace

@git 으로 git 서버 전체,
@git/status 로 특정 도구.
@builtin 은 모든 내장 도구.

WILDCARD

전체 매칭

* 로 모든 tool
(built-in + MCP).
matcher 생략도 동일하게 모두
매칭.

matcher

필드 이름

Aliases

단축 별칭

@server

MCP prefix

*

전체 wildcard

Timeout 과 Caching

PERFORMANCE OPTIMIZATION

TIMEOUT & CACHING

Hook 실행에는 두 가지 성능 설정.

`timeout_ms` 는 최대 실행 시간 (기본 30초), `cache_ttl_seconds` 는 결과 캐싱 시간 (기본 0, 캐싱 안 함). AgentSpawn 은 캐싱 불가.

TIMEOUT_MS

실행 시간 제한

1. 기본값 30,000 ms (30 초)
2. 느린 hook 이 chat 흐름 차단 방지
3. Timeout 초과 시 hook 실패 처리
4. 필요 시 명시적 `timeout_ms` 설정

CACHE_TTL_SECONDS

결과 캐싱 옵션

1. 기본 0, 캐싱 비활성화
2. 양수 값, 그 시간 동안 결과 재사용
3. 동일 hook 의 반복 실행 비용 절약
4. AgentSpawn 은 never cached, 항상 실행

30 s

기본 timeout

`timeout_ms`

설정 필드

0

기본 cache TTL

Never

AgentSpawn 캐싱

활용 예시와 Best Practices

REAL-WORLD HOOK PATTERNS

EXAMPLES

흔한 활용 예시 4 가지

1. Audit log, execute_bash 시 명령 기록
2. AWS audit, use_aws 호출 별도 기록
3. Auto format, fs_write 후 prettier 자동
4. Git context, agentSpawn 시 status 수집

BEST PRACTICES

효과적인 hook 작성

1. Fast execution, 빠른 명령 사용 권장
2. Idempotent, 동일 입력 동일 결과
3. Specific matcher, 필요한 도구만 매칭
4. Test thoroughly, 차단 hook 은 특히 검증

Audit

Logging 패턴

Format

PostToolUse 활용

Fast

Hook 성능 원칙

Idempotent

안전 동작 원칙

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

